Università degli Studi di Verona

Corso di Laurea in Bioinformatica

Algoritmi per Bioinformatica

e Laboratorio di Programmazione II

I seguenti appunti sono preparati tramite i lucidi del professore e il libro di riferimento

Per la parte di laboratorio il professore prende gli esercizi da:
`https://leetcode.com/problemset/algorithms/`

Anno Accademico 2025/2026

Indice

1 Fondamenti **3**

1.1 Introduzione al corso . . . 3

1.1.1 Requisiti . . . 3

1.1.2 Programma del corso . . . 3

1.1.3 Struttura del modulo . . . 3

1.1.4 Modalità di esame . . . 3

1.2 Il ruolo degli algoritmi nell'informatica . . . 3

1.2.1 Cos'è un algoritmo? . . . 3

1.3 Esempio di algoritmo ricorsivo . . . 4

1.4 Esempi di problemi computazionali da risolvere in aula . . . 7

1.5 InsertionSort e MergeSort . . . 12

1.5.1 InsertionSort . . . 12

1.6 Tasso di crescita e ordine di grandezza . . . 17

1.6.1 MergeSort (Divide et impera) . . . 17

1.7 Analisi probabilistica e algoritmi randomizzati . . . 18

1.7.1 Il problema dell'assunzione . . . 18

1.7.2 Variabili casuali indicatrici . . . 18

1.7.3 Algoritmi randomizzati . . . 18

1.8 Calcolo della complessità . . . 18

1.8.1 Notazione asintotica . . . 18

1.8.2 Regole per algoritmi iterativi . . . 18

1.8.3 Ricorrenze: forma generale . . . 19

1.8.4 Metodo della sostituzione . . . 19

1.8.5 Metodo dell'albero di ricorsione . . . 19

1.8.6 Master Theorem . . . 19

1.8.7 Ricorrenze non standard (telescoping) . . . 20

2 Può tornare utile **20**

2.1 Appendice A (Sommatorie) . . . 20

2.2 Appendice B (Insiemi, relazioni, funzioni, grafi, alberi) . . . 20

2.3 Appendice C (Calcolo combinatorio e probabilità) . . . 20

2.4 Appendice D (Matrici) . . . 20

2.5 Domande d'esame (Appello 2) . . . 20

2.5.1 Definizione di algoritmo . . . 20

2.5.2 Problema computazionale . . . 21

2.5.3 Efficienza . . . 21

2.5.4 InsertionSort vs MergeSort . . . 21

2.5.5 $100n^2$ vs 2^n . . . 21

2.5.6 Invariante di ciclo . . . 21

2.5.7 Vale $\max(f, g) = \Theta(f + g)$? . . . 21
2.5.8 2^{n+1} e 2^{2n} vs $O(2^n)$ . . . 21
2.5.9 Costo costante in RAM . . . 21
2.5.10 Prestazioni su istanze note . . . 21
2.5.11 $T(n) = T(n-1) + n^c$ . . . 21
2.5.12 $T(n) = 2T(n/3) + 1$ . . . 21
2.5.13 Divide et impera: classe di problemi . . . 22
2.5.14 Analisi probabilistica . . . 22
2.5.15 permute-without-identity . . . 22
2.5.16 Hire-Assistant: probabilità . . . 22
2.5.17 Generatore casuale . . . 22
2.5.18 Algoritmo randomizzato . . . 22
2.5.19 Ricorrenza con frazioni . . . 22
2.5.20 Partition su array . . . 22
2.5.21 Quicksort con elementi uguali . . . 22
2.5.22 Partition elementi uguali: fix . . . 22
2.5.23 Quicksort non crescente . . . 22
2.5.24 Rand-Select: caso peggiore . . . 22
2.5.25 IV statistica d'ordine . . . 23
2.5.26 Eliminare duplicati . . . 23
2.5.27 Build-Max-Heap: indice decrescente . . . 23
2.5.28 Visita inorder con pila . . . 23
2.5.29 LCS senza tabella b . . . 23
2.5.30 Cammino minimo vs TSP . . . 23
2.5.31 Prima e ultima cifra (Java) . . . 23
2.5.32 `T extends Comparable` . . . 23
2.5.33 Ereditarietà e attributo omonimo . . . 23
2.5.34 Errore in makeCombination . . . 24
2.5.35 Eccezione in try-with-resources . . . 24
2.5.36 Downcasting Java . . . 24
2.5.37 Stack con due code . . . 24
2.5.38 Ricerca in matrice destrorsa . . . 24

1. Fondamenti

1.1 Introduzione al corso

1.1.1 Requisiti

- Fondamenti di matematica discreta e grafi
- Calcolo delle probabilità
- Basi di C → si studia Java (Classi e Oggetti)[1]
- Le procedure ricorsive sono come l'Ave Maria (STUDIA: c'è nell'esame)

1.1.2 Programma del corso

- Divide et Impera
- Programmazione dinamica
- Algoritmi Greedy
- Grafi → Nodi e archi

1.1.3 Struttura del modulo

48 ore di lezione: 4 ore a settimana + 9 ore di studio personale.[2] Non usare ChatGPT o altri strumenti simili.

I lucidi non bastano (purtroppo).

Se proprio devi, usa **Anthropic CLAUDE**.

1.1.4 Modalità di esame

- **Prima parte:** 15 domande a risposta multipla (almeno 4 su codice Java – CODE RUNNER)
- **Seconda parte:** Progettare una soluzione algoritmica
- **Se va male:** Può fare orale se ha dubbi (Ave o Maria...)

1.2 Il ruolo degli algoritmi nell'informatica

1.2.1 Cos'è un algoritmo?

Definizione 1.1 (Algoritmo)**.** Procedura di calcolo ben definita: dato un input (anche nullo), produce un output in tempo finito.

[1] Java è il male

[2] Sì, lo so, è una noia, ma serve.

Definizione 1.2 (Problema computazionale)**.** Relazione input/output desiderata. L'algoritmo specifica *come* ottenerla.

Definizione 1.3 (Algoritmo corretto)**.** Termina producendo il risultato corretto per ogni istanza valida.

Osservazione 1. Non sempre esiste un algoritmo: il problema della fermata è indecidibile.

1.3 Esempio di algoritmo ricorsivo

Nota

Le convenzioni usate per il pseudocodice sono a pag. 19 del [CLRS].

Algorithm 1: FibonacciR

Input: $n \geq 0$

Output: F_n

```
if n = 0 then
    return 0;
if n = 1 then
    return 1;
return FibonacciR(n − 1) + FibonacciR(n − 2);
```

L'algoritmo è corretto? **Sì!**

Per ogni input valido, termina? **Sì, assumendo che input sia sempre un intero n $\geq$ 0**

Se sì, calcola il valore giusto? **Sì, perché l'algoritmo codifica la definizione ricorsiva.**

Quanto **tempo** richiede per terminare?

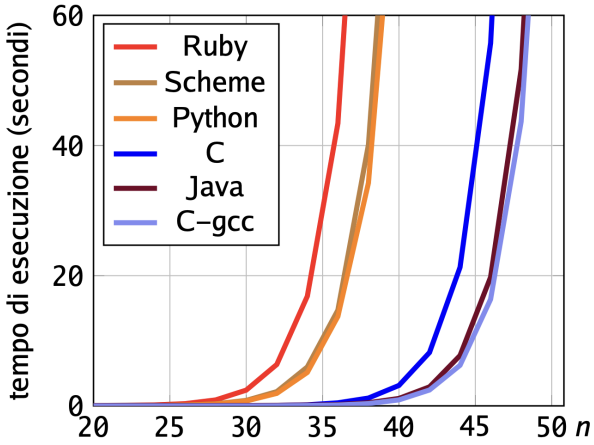


Figura 1. Tempo computazionale dell'algoritmo `FibonacciR`

Come mostrato in Figura 1, il tempo cresce esponenzialmente.

Calcolo della complessità computazionale

Definizione 1.4 (Efficienza)**.** Risorse (tempo e spazio) richieste in funzione della dimensione n dell'input.

Definizione 1.5 (Algoritmo vs. Implementazione)**.** L'algoritmo è un concetto **astratto** ($\neq$ implementazione): descrive *come* risolvere un problema indipendentemente dal linguaggio. L'implementazione è un **programma concreto**, ovvero la traduzione dell'algoritmo in un linguaggio comprensibile dalla

$$T(n) = \begin{cases} mac & \text{se } condizione \\ caso_r icorsivo & \text{altrimenti} \end{cases}$$

china.

Nozione 1.1 (Algoritmo efficiente)**.** Andiamo a cercare l'algoritmo efficiente:

- **Efficiente:** usa poche risorse
- **Risorse:** tempo e spazio richiesti dall'algoritmo per produrre l'output

Teorema 1.1 (Algoritmo vs. hardware)**.** *Con $n = 10^7$ interi da ordinare:*

$$t_A = \frac{2(10^7)^2}{10^{10}} \approx 5.5 \ ore \quad (InsSort,\ 10^9\ istr/s)$$
$$t_B = \frac{50 \cdot 10^7 \log_2 10^7}{10^6} \approx 20 \ min \quad (MergeSort,\ 10^6\ istr/s)$$

L'algoritmo migliore batte l'hardware più veloce.

Possiamo fare meglio?

Ottimizzazione: FibonacciI

Algorithm 2: FibonacciI

Input: Intero positivo n

Output: Il valore F_n

```
if n = 0 then
    return 0;
if n = 1 then
    return 1;
pprev = 0;
prev = 1;
for i = 2 to n do
    f = prev + pprev;
    pprev = prev;
    prev = f;
return f;
```

La sua complessità soddisfa la ricorrenza:

$$T(0) = 2, \quad T(1) = 3, \quad T(n) = T(n-1) + T(n-2) + 8$$

Il +8 è dato da: 2 `if` + 2 calcoli parametri + 2 chiamate + 1 somma + 1 `return`. Da questa ricorrenza segue $T(n) > F_n$, e per $n \geq 7$:

$$F_n > 2F_{n-2} > 2^2 F_{n-4} > \cdots > 2^{n/2}$$

Quindi la crescita è **esponenziale**:

$$T(n) > \left(\sqrt{2}\right)^n \approx (1.4)^n$$

Una stima più precisa è data dalla **Formula di Binet**:

$$T(n) \geq \frac{\varphi^n - (-\varphi)^{-n}}{\sqrt{5}}, \qquad \varphi = \frac{1+\sqrt{5}}{2} \approx 1.62$$

L'algoritmo iterativo evita il ricalcolo sommando i valori dal più piccolo al più grande. La complessità è:

$$T(0) = 2, \quad T(1) = 3, \quad T(n) = 4 + 2n + 4(n-1) + 1 = 6n + 1$$

La crescita è **lineare in** n.

Algoritmo	Complessità	$n = 50$
Ricorsivo	$(1.4)^n$	minuti
Iterativo	$\Theta(n)$	istantaneo

Modello RAM: ogni operazione elementare ha costo 1; parola di memoria a dimensione fissa.

1.4 Esempi di problemi computazionali da risolvere in aula

Iindici dei due numeri che sommati danno N

Problema: Data una matrice di interi `nums` e un intero `target`, restituisci gli indici dei due numeri tali che la loro somma sia uguale a `target`. Puoi assumere che ogni input abbia esattamente una soluzione, e non puoi usare lo stesso elemento due volte. Puoi restituire la risposta in qualsiasi ordine.

Algorithm 3: twoSum (Brute Force)

Input: `nums[0...n-1]`, `target`

Output: {i, j} tali che `nums[i] + nums[j] = target`, $i \neq j$

```
for i = 0 to n − 2 do
    for j = i + 1 to n − 1 do
        if nums[i] + nums[j] == target then
            return {i, j};
```

Invariante: Dopo ogni iterazione esterna, tutti gli elementi da 0 a $i - 1$ sono stati confrontati con tutti gli elementi successivi.

Complessità:

- Caso migliore: $\Theta(1)$ (se la coppia è trovata subito)
- Caso peggiore: $\Theta(n^2)$

Calcolo della complessità temporale (caso peggiore):

L'algoritmo esegue due cicli annidati. Il ciclo esterno itera $n - 1$ volte, e per ogni i il ciclo interno itera $n - 1 - i$ volte. Quindi:

$$T(n) = \sum_{i=0}^{n-2}(n - 1 - i) = \sum_{k=1}^{n-1} k = \frac{(n-1)n}{2} \in \Theta(n^2)$$

La complessità spaziale è $\Theta(1)$, poiché usiamo solo variabili costanti.

Codice JAVA

```java
class twoSum {
    public int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{};  // nessuna soluzione trovata
    }
}
```

Problema dell'input

Algorithm 4: Inversione numerica

Input: Intero x rappresentato con segno a 32 bit

Output: Intero y con le cifre di x invertite

```
y = 0;                                  // Inizializza risultato
sign = 1;                               // Memorizza segno
sign = (x < 0)? − 1 : 1;                // Memorizza segno di x
x = |x|;                                // Rende x positivo
while x > 0 do
    digit = x mod 10;                   // Estrae ultima cifra
    y = y × 10 + digit;                 // Aggiunge a y
    x = ⌊x/10⌋;                         // Rimuove cifra
return y × sign;                        // Restituisce con segno
```

Complessità. Sia k il numero di cifre decimali di $|x|$. Con $c = 6$ Allora

$$T(k) = \begin{cases} 6 & \text{se } x = 0, \\ 5k + 6 & \text{se } x > 0, \\ 5k + 8 & \text{se } x < 0, \end{cases} \qquad \Rightarrow \qquad T(k) = \mathcal{O}(k).$$

$$T = 3 + (\text{n. cicli}) + 12 \cdot (\text{n. cicli}) + 2$$

La complessità dell'algoritmo è lineare.

Nota

L'**operatore ternario** è definito mome: x = (espressione booleana) ? v1 : v2 se l'espressione booleana è vera viene usata v1 se l'espressione booleana è falsa viene usata v2

Container con più acqua

Si consideri un array di interi non negativi *height* di lunghezza n. Per ogni i, la retta verticale ha estremi $(i, 0)$ e $(i, height[i])$. Si vogliono trovare due rette che, insieme all'asse x, formino un contenitore che possa contenere la massima quantità d'acqua.

Nel nostro esempio poniamo

$$A = [4, 6, 3, 5].$$

L'area del contenitore formato dalle rette in posizione i e j $(i < j)$ è data da

$$\text{area}(i, j) = (j - i)\ \min(A[i], A[j]).$$

Per $A = [4, 6, 3, 5]$ otteniamo:

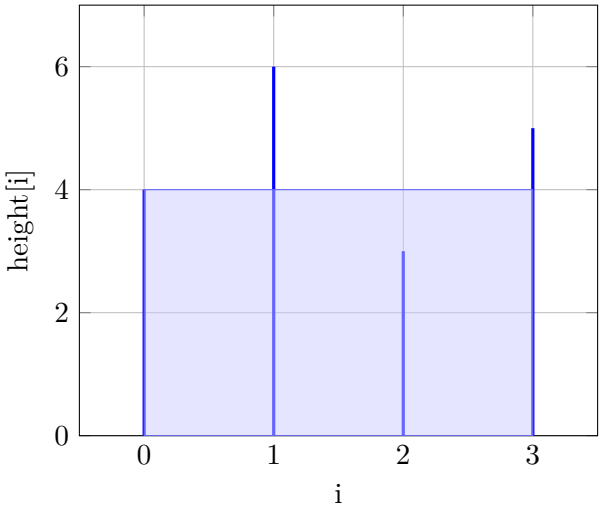


Figura 2. Container con area massima per $A = [4, 6, 3, 5]$.

Il valore massimo è quindi

$$\max_{0 \le i < j \le 3} \text{area}(i, j) = 12,$$

ottenuto per la coppia di indici $(i, j) = (0, 3)$.

Quindi il pseudocodice per questo problema computazionale è il seguente:

Algorithm 5: Container With Most Water

Input: Array $height[0 \dots n-1]$ di interi non negativi

Output: Massima area d'acqua contenibile

```
left ← 0;                                          // Puntatore sinistro
right ← n − 1;                                     // Puntatore destro
maxArea ← 0;                                       // Area massima trovata finora
while left < right do
    width ← right − left;                          // Larghezza del contenitore
    h ← min(height[left], height[right]);          // Altezza = lato più basso
    area ← width · h;                              // Area del contenitore corrente
    maxArea ← max(maxArea, area);                  // Aggiorna massimo
    if height[left] < height[right] then
        left ← left + 1;                           // Sposta il lato più basso verso il centro
    else
        right ← right − 1;                         // Sposta il lato più basso verso il centro
return maxArea;                                    // Restituisce l'area massima
```

Complessità. Ad ogni iterazione del ciclo `while` viene spostato solo uno dei due puntatori, che possono muoversi ciascuno al più $n-1$ volte. Pertanto il numero di iterazioni è al più $n-1$ e il costo è

$$T(n) = c_1 + c_2(n-1) = \Theta(n).$$

Lo spazio aggiuntivo utilizzato è costante, quindi $S(n) = \Theta(1)$.

Prefisso piu lungo comune fra più sequenze

Algorithm 6: Prefisso comune più lungo

Input: Array $S[1 \dots n]$ di stringhe

Output: Prefisso più lungo comune a tutte le stringhe

```
// Assumo che le stringhe di S siano tutte diverse da null
p ← S[1];                                   // Prefisso corrente
for i ← 2 to n do
    p ← PrefissoComune(p, S[i]);            // Prefisso tra p e S[i]
    if p = stringa vuota then
        return p;                           // Nessun prefisso comune
return p;                                   // Prefisso comune più lungo
```

Complessità. Sia n il numero di stringhe in S e sia m la lunghezza della stringa più corta. Ad ogni passo del ciclo la funzione `PrefissoComune` confronta al più m caratteri, quindi

$$T(n, m) = c_1 + c_2(n - 1)m = \Theta(nm).$$

Lo spazio aggiuntivo utilizzato è costante, dunque $S(n, m) = \Theta(1)$.

1.5 InsertionSort e MergeSort

1.5.1 InsertionSort

Inserisce ogni elemento nella posizione corretta tra quelli già ordinati (come le carte da gioco).

Algorithm 7: InsertionSort

Input: $A = \langle a_1, \ldots, a_n \rangle$, array di elementi confrontabili con $\leq$
Output: A ordinato rispetto a $\leq$

```
for i = 2 to n do
    key = A[i] ;            // Inserisce A[i] nella parte ordinata a_1,...,a_{i-1}
    j = i - 1;
    while j > 0 and A[j] > key do
        A[j + 1] = A[j];
        j = j - 1;
    A[j + 1] = key;
return A
```

Nota

Ordina gli elementi **sul posto**.

Ciclo Algoritmo

5	2	4	6	1	$= A$

Ogni blocco rappresenta un ciclo for (0 o più cicli while)

1	*2*	*3*	*4*	*5*	*Commento*
5 j	2 i	4	6	1	Seq. ord.: 5 Scambio 5→2
2 ←	5 j	4 i	6	1	Seq. ord.: 2,5 Scambio 5→4
2	4 ←	5 j	6 i	1	Seq. ord.: 2,4,5 Nessun scambio
2	4	5 ←	6 j	1 i	Seq. ord.: 2,4,5,6 1 → pos. I
1	2	4	5	6 i	Seq. ordinata

Invariante di ciclo

Definizione 1.6 (Invariante di ciclo)**.** Un'asserzione (proprietà booleana) sempre vera all'inizio di ogni iterazione di un ciclo.

La validità si dimostra in tre passi:

1. **Inizializzazione:** è vera prima della prima iterazione.
2. **Conservazione:** se è vera prima di un'iterazione, rimane vera prima di quella successiva.
3. **Conclusione:** quando il ciclo termina, l'invariante fornisce una proprietà utile a dimostrare la correttezza.

Esempio 1.1 (Invariante di InsertionSort). *"All'inizio del ciclo* `for`*, il sottoarray* $A[1 : i-1]$ *è ordinato e formato dagli stessi elementi che erano in* $A[1 : i-1]$*."*

- **Inizializzazione:** $i = 2$, quindi $A[1 : 1]$ è un solo elemento $\Rightarrow$ ordinato. ✓
- **Conservazione:** `key`$= A[i]$, il ciclo `while` sposta $A[i-1], A[i-2], \ldots$ a destra finché non trova la posizione corretta per `key`. Quindi $A[1 : i]$ diventa ordinato con gli stessi elementi. All'iterazione successiva i aumenta di 1, l'invariante vale su $A[1 : i-1]$. ✓
- **Conclusione:** il ciclo termina con $i = n + 1$. Per l'invariante, $A[1 : n]$ è ordinato con gli stessi elementi $\Rightarrow$ l'algoritmo è corretto. ✓

Modello RAM

Per analizzare la complessità si usa un modello astratto di computer: la **Random Access Machine (RAM)**.

- Un solo processore con memoria ad accesso casuale.
- Numeri interi e a virgola mobile con dimensione fissa (es. 64 bit per parola).
- Istruzioni elementari (lettura, scrittura, somma, sottrazione, moltiplicazione, divisione, confronto, `if`, salto, chiamata a procedura).
- Ogni istruzione ha **costo costante** $= 1$.

Nota

È una semplificazione: nella realtà cache, pipeline e parallelismo influenzano i tempi reali. In questo corso si ignorano questi dettagli.

Dimensione dell'input e tempo di esecuzione

Definizione 1.7 (Dimensione dell'input)**.** Misura della quantità di bit/word necessari per rappresentare l'input. Un indicatore è **corretto** se la dimensione reale è in relazione polinomiale con esso.

Definizione 1.8 (Tempo di esecuzione)**.** Numero totale di operazioni elementari eseguite dall'algoritmo nel modello RAM, in funzione della dimensione n dell'input.

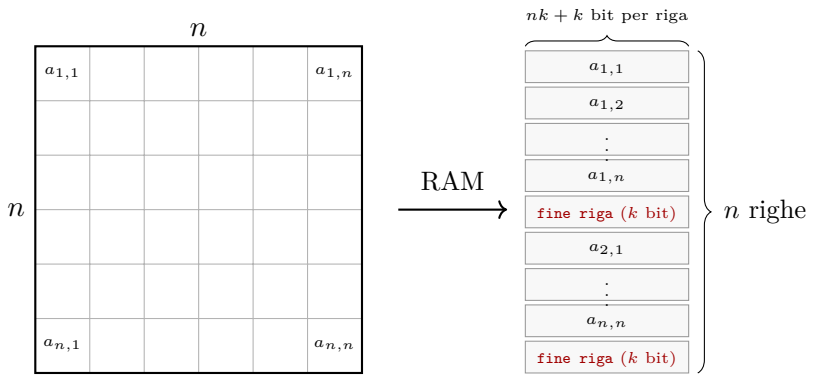
Esempi di indicatori corretti:

- **Array di n elementi:** indicatore $= n$. ✓
- **Matrice $n \times n$:** indicatore $= n$ (la dimensione reale $n \cdot k(n+1)$ è polinomiale in n). ✓
- **Intero n:** indicatore $= \lfloor \lg n \rfloor + 1$ (numero di bit). **Non** si può usare n come indicatore perché $2^{|I|} \approx n$, relazione esponenziale. ✓

Esempio 1.2 (Matrice $n \times n$). Sia A una matrice $n \times n$ di interi, dove ogni intero richiede k bit. La RAM memorizza la matrice riga per riga.

Ogni riga occupa nk bit per gli interi, più k bit per il marcatore di fine riga. L'intera matrice richiede:

$$|I| = n(n+1)k \text{ bit} = n(kn+k) \text{ bit} = \boxed{kn^2 + kn \text{ bit}} = \Theta(n)$$



Si usa come indicatore $|I| = n$. È **corretto** perché la dimensione reale $kn^2 + kn$ è un **polinomio in** n.

Nota

L'errore comune è usare il *valore* di un intero n come dimensione dell'input invece del numero di bit necessari a rappresentarlo. Questo porta a sottostimare la complessità.

Definizione 1.9 (Tempo di esecuzione)**.** Il tempo di esecuzione di un algoritmo per un dato input di dimensione n è il numero totale di operazioni elementari eseguite (passi) nel modello RAM.

Analisi di InsertionSort

c_k = costo esecuzione istruzione alla riga k. $t_i - 1 = \#$ cicli **while** nel i-esimo ciclo **for**.

	INSERTION-SORT(A, n)	*cost*	*times*
1	**for** $i = 2$ **to** n	c_1	n
2	$key = A[i]$	c_2	$n-1$
3	`// Insert` $A[i]$ `into the sorted subarray` $A[1:i-1]$	0	$n-1$
4	$j = i - 1$	c_4	$n-1$
5	**while** $j > 0$ **and** $A[j] > key$	c_5	$\sum_{i=2}^{n} t_i$
6	$A[j+1] = A[j]$	c_6	$\sum_{i=2}^{n}(t_i - 1)$
7	$j = j - 1$	c_7	$\sum_{i=2}^{n}(t_i - 1)$
8	$A[j+1] = key$	c_8	$n-1$

Contare le iterazioni:
for $i = \ell$ **to** n $\Rightarrow$ $\#\text{Cicli} = n - \ell + 1$
Esempio: for $i = 2$ **to** n $\Rightarrow$ $n - 1$ iterazioni

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^{n} t_i + c_6 \sum_{i=2}^{n}(t_i - 1) + c_7 \sum_{i=2}^{n}(t_i - 1) + c_8(n-1)$$

Indichiamo con c_k il costo dell'istruzione alla riga k, e con t_i il numero di volte che il ciclo `while` viene eseguito durante l'i-esimo ciclo `for`.

Il tempo totale è:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^{n} t_i + c_6 \sum_{i=2}^{n}(t_i - 1) + c_7 \sum_{i=2}^{n}(t_i - 1) + c_8(n-1)$$

Caso migliore: array già ordinato

Se l'array è già ordinato, il corpo del `while` non viene mai eseguito: $t_i = 1$ per ogni i. Le somme $\sum(t_i - 1) = 0$, quindi:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= \underbrace{(c_1 + c_2 + c_4 + c_5 + c_8)}_{a} n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_{b} \\ &= an - b \quad \Rightarrow \quad \boxed{T(n) = \Theta(n)} \end{aligned}$$

Nota

Nel caso migliore la crescita è **lineare**: ogni elemento viene confrontato una sola volta con il suo predecessore e nessuno scambio viene effettuato.

Caso peggiore: array ordinato al contrario

Se l'array è in ordine decrescente, ogni $A[i]$ deve essere confrontato con **tutti** gli elementi in $A[1 : i-1]$, quindi $t_i = i$. Usando $\sum_{i=2}^{n} i = \frac{n(n+1)}{2} - 1$:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + (c_6 + c_7)\frac{n(n-1)}{2} + c_8(n-1)$$

Raccogliendo i termini per grado:

$$T(n) = \underbrace{\frac{c_5 + c_6 + c_7}{2}}_{a} n^2 + \underbrace{\left(c_1 + c_2 + c_4 + \frac{c_5 - c_6 - c_7}{2} + c_8\right)}_{b} n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_{c}$$

$$\Rightarrow \quad \boxed{T(n) = \Theta(n^2)}$$

Nota

Nel caso peggiore la crescita è **quadratica**: ogni nuovo elemento deve "scendere" fino in fondo alla parte già ordinata, confrontandosi con tutti gli elementi precedenti.

Caso	Input	Complessità
Migliore	Array già ordinato	$\Theta(n)$
Peggiore	Array ordinato inverso	$\Theta(n^2)$

Solitamente si considera come complessità di un algoritmo il tempo di esecuzione con input che rappresenta il caso peggiore.

1. È una garanzia sulla descrizione del comportamento dell'algoritmo.
2. In alcuni casi, il caso peggiore è il più frequente.
3. Il caso medio spesso è simile al caso peggiore (vedi InsertionSort).

1.6 Tasso di crescita e ordine di grandezza

Nell'analisi di InsertionSort si sono usate costanti c_k per rappresentare i costi astratti delle operazioni nel modello RAM. Queste costanti possono essere raggruppate: ad esempio $(c_2 + c_4 + c_5 + c_8)$ diventa una costante generica a.

Ultima astrazione: è più importante valutare la **velocità con cui cresce** il tempo di esecuzione (*tasso di crescita*) che i dettagli delle costanti.

Di un polinomio $T(n) = an^2 + bn + c$ è sufficiente considerare solo il termine dominante an^2. Visto che la costante a è astratta, la si assorbe nell'ordine di grandezza:

$$T(n) = an^2 + bn + c \approx an^2 \implies \Theta(n^2)$$

Un algoritmo è considerato **più efficiente** di un altro se il suo tempo di computazione nel caso peggiore ha un **tasso di crescita inferiore**.

$T(n)$	Ordine di grandezza	Nome
$an + b$	$\Theta(n)$	Lineare
$an^2 + bn + c$	$\Theta(n^2)$	Quadratica
$an^3 + \cdots$	$\Theta(n^3)$	Cubica
$a \lg n + b$	$\Theta(\lg n)$	Logaritmica
$an \lg n + b$	$\Theta(n \lg n)$	Linearitmica

1.6.1 MergeSort (Divide et impera)

1. **Dividi:** spezza in due metà.
2. **Impera:** ordina ricorsivamente.
3. **Combina:** fondi in $\Theta(n)$.

$$T(n) = 2\,T(n/2) + \Theta(n) \;\Rightarrow\; T(n) = \Theta(n \log n)$$

Algoritmo	Caso peggiore	Strategia
InsertionSort	$\Theta(n^2)$	Incrementale
MergeSort	$\Theta(n \log n)$	Divide et impera

1.7 Analisi probabilistica e algoritmi randomizzati

1.7.1 Il problema dell'assunzione

1.7.2 Variabili casuali indicatrici

1.7.3 Algoritmi randomizzati

1.8 Calcolo della complessità

1.8.1 Notazione asintotica

Definizione 1.10 (O, Ω, Θ)**.** Date $f, g : \mathbb{N} \to \mathbb{R}^+$:

- $f = O(g)$: $\exists\, c, n_0$ t.c. $f(n) \leq c\, g(n)$ per $n \geq n_0$ (*limite superiore*).
- $f = \Omega(g)$: $\exists\, c, n_0$ t.c. $f(n) \geq c\, g(n)$ per $n \geq n_0$ (*limite inferiore*).
- $f = \Theta(g)$: $f = O(g)$ e $f = \Omega(g)$ (*limite stretto*).

Ordine crescente:

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(2^n) \subset O(n!)$$

1.8.2 Regole per algoritmi iterativi

- **Sequenza:** $O(\max(f, g))$.
- **Ciclo semplice** di n iterazioni a corpo $O(1)$: $O(n)$.
- **Cicli annidati:** prodotto dei range. Due cicli $1 \ldots n$: $O(n^2)$.
- **Passo moltiplicativo** (i raddoppia): $O(\log n)$.
- **If-else:** massimo dei due rami.

Esempio (limite variabile):

```
for (int i = 1; i <= n; i++)
for (int j = 1; j <= i; j++)
op(); // O(1)
```

Costo: $\sum_{i=1}^{n} i = \dfrac{n(n+1)}{2} = \Theta(n^2)$.

1.8.3 Ricorrenze: forma generale

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n), \quad T(1) = \Theta(1)$$

a = numero sotto-chiamate, b = fattore riduzione, $f(n)$ = costo divisione + combinazione.

1.8.4 Metodo della sostituzione

1. **Ipotizzare** la forma, es. $T(n) = O(n \log n)$.
2. **Sostituire** nella ricorrenza assumendo l'ipotesi vera per $T(n/b)$.
3. **Verificare** la disuguaglianza scegliendo le costanti.

Esempio: $T(n) = 2T(n/2) + n$, ipotesi $T(n) \le cn \log n$:

$$T(n) \le 2 \cdot \frac{cn}{2} \log \frac{n}{2} + n = cn \log n - cn + n \le cn \log n \quad (c \ge 1).$$

1.8.5 Metodo dell'albero di ricorsione

1. Ogni nodo al livello k ha costo $f(n/b^k)$ e genera a figli.
2. Numero livelli: $\log_b n$. Foglie: $n^{\log_b a}$.
3. Sommare i costi per livello.

Esempio: $T(n) = 3T(n/4) + cn^2$. Costo per livello k: $c\,n^2(3/16)^k$.

$$T(n) < cn^2 \sum_{k=0}^{\infty} \left(\frac{3}{16}\right)^k = cn^2 \cdot \frac{16}{13} = O(n^2).$$

1.8.6 Master Theorem

Teorema 1.2 (Master Theorem)**.** *Data $T(n) = a\,T(n/b) + f(n)$, posto $p = \log_b a$:*

1. $f(n) = O(n^{p-\varepsilon}) \Rightarrow T(n) = \Theta(n^p)$ (foglie dominano)
2. $f(n) = \Theta(n^p \log^k n)$, $k \ge 0 \Rightarrow T(n) = \Theta(n^p \log^{k+1} n)$ (pareggio)
3. $f(n) = \Omega(n^{p+\varepsilon})$ *e* $a\,f(n/b) \le c\,f(n)$, $c < 1 \Rightarrow T(n) = \Theta(f(n))$ (radice domina)

Esempi applicativi:

Ricorrenza	$p = \log_b a$	Caso	Soluzione
$2T(n/2) + \Theta(n)$	1	2	$\Theta(n \log n)$
$2T(n/2) + \Theta(n^2)$	1	3	$\Theta(n^2)$
$4T(n/2) + \Theta(n)$	2	1	$\Theta(n^2)$
$2T(n/3) + \Theta(1)$	$\log_3 2$	1	$\Theta(n^{\log_3 2})$
$T(n/2) + \Theta(1)$	0	2	$\Theta(\log n)$

Osservazione 2. Il Master Theorem **non si applica** se $f(n)$ non è polinomialmente confrontabile con n^p o se la forma non è $a\,T(n/b) + f(n)$.

1.8.7 Ricorrenze non standard (telescoping)

Ricorrenza	Soluzione
$T(n) = T(n-1) + O(1)$	$\Theta(n)$
$T(n) = T(n-1) + O(n^c)$	$\Theta(n^{c+1})$
$T(n) = 2T(n-1) + O(1)$	$\Theta(2^n)$
$T(n) \leq T(\alpha n) + T((1-\alpha)n) + O(n)$	$O(n \log n)$

Telescoping: $T(n) = T(n-1) + f(n) = T(1) + \sum_{k=2}^{n} f(k)$. Per $f(k) = k^c$: somma $= \Theta(n^{c+1})$.

2. Può tornare utile

2.1 Appendice A (Sommatorie)

2.2 Appendice B (Insiemi, relazioni, funzioni, grafi, alberi)

2.3 Appendice C (Calcolo combinatorio e probabilità)

2.4 Appendice D (Matrici)

2.5 Domande d'esame (Appello 2)

2.5.1 Definizione di algoritmo

D: Definizione sufficiente di algoritmo?
R: Procedura ben definita che produce output da input in tempo finito.

2.5.2 Problema computazionale

D: Cos'è un problema computazionale?
R: Relazione input-output desiderata. Il problema dice *cosa*, l'algoritmo dice *come*.

2.5.3 Efficienza

D: Cosa si intende per efficienza?
R: Come crescono le risorse al crescere di n (non il tempo in secondi, dipendente dall'hardware).

2.5.4 InsertionSort vs MergeSort

D: InsSort $= 10n^2$, MergeSort $= 32n \log_2 n$. Quando è più veloce InsSort?
R: Per n piccoli: $10n^2 < 32n \log_2 n$ solo inizialmente.

2.5.5 $100n^2$ vs 2^n

D: Minimo n tale che $100n^2 < 2^n$?
R: $n \geq 15$.

2.5.6 Invariante di ciclo

D: Cos'è un'invariante di ciclo?
R: Proprietà vera all'inizio di ogni iterazione. Si dimostra per inizializzazione, conservazione e conclusione.

2.5.7 Vale $\max(f, g) = \Theta(f + g)$?

R: Sì: $\max(f, g) \leq f + g \leq 2 \max(f, g)$, con $c_1 = 1$, $c_2 = 2$.

2.5.8 2^{n+1} e 2^{2n} vs $O(2^n)$

R: $2^{n+1} = O(2^n)$ (costante 2). $2^{2n} = 4^n \neq O(2^n)$ (richiederebbe $2^n \leq c$, impossibile).

2.5.9 Costo costante in RAM

R: Quando la parola di memoria ha dimensione fissa (es. 64 bit).

2.5.10 Prestazioni su istanze note

R: Precomputare le soluzioni e restituirle direttamente quando quell'istanza viene ricevuta.

2.5.11 $T(n) = T(n-1) + n^c$

R: $T(n) = \Theta(n^{c+1})$. Dim.: $\sum_{k=1}^{n} k^c \approx \int_1^n x^c \, dx = \frac{n^{c+1}}{c+1}$.

2.5.12 $T(n) = 2T(n/3) + 1$

R: Master Theorem, caso 1: $T(n) = \Theta(n^{\log_3 2})$ ($\log_3 2 \approx 0.63$).

2.5.13 Divide et impera: classe di problemi

R: Problemi divisibili in 2+ sotto-istanze con combinazione a costo al più polinomiale.

2.5.14 Analisi probabilistica

R: Uso del calcolo della probabilità nell'analisi degli algoritmi (non solo caso medio).

2.5.15 permute-without-identity

R: No: con $n = 3$ produce solo 3 permutazioni invece delle 5 attese.

2.5.16 Hire-Assistant: probabilità

R: Esattamente 1 assunzione: $1/n$. Esattamente n assunzioni: $1/n!$.

2.5.17 Generatore casuale

R: Valore intero in un intervallo con probabilità uniforme.

2.5.18 Algoritmo randomizzato

R: Il comportamento dipende dall'input e da valori casuali.

2.5.19 Ricorrenza con frazioni

D: $T(n) \leq T(10n/11) + T(n/11) + O(n)$?
R: $T(n) = O(n \log n)$. Ogni livello costa $O(n)$, profondità $O(\log n)$.

2.5.20 Partition su array

D: Partition su $[13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$, pivot $A[11]$?
R: $[9, 5, 8, 7, 4, 2, 6, \mathbf{11}, 21, 13, 19, 12]$, pivot in pos. 7.

2.5.21 Quicksort con elementi uguali

R: $O(n^2)$: ogni Partition genera partizione $(n-1)$ vs 0.

2.5.22 Partition elementi uguali: fix

R: Restituisce indice massimo. Fix: flag booleano; se tutti uguali, restituire $\lfloor (p + r)/2 \rfloor$.

2.5.23 Quicksort non crescente

R: Invertire il confronto in Partition: da $\leq$ a $\geq$.

2.5.24 Rand-Select: caso peggiore

R: Il pivot è sempre il massimo: dimensione scende di 1 a ogni passo, $O(n^2)$.

2.5.25 IV statistica d'ordine

D: IV statistica di $[8, 4, 8, 3, 4, 2, 3, 2]$?
R: Ordinato: $[2, 2, 3, 3, 4, 4, 8, 8]$. IV elemento $= 3$.

2.5.26 Eliminare duplicati

D: Eliminare duplicati in $\Theta(n \log n)$?
R: Ordinare, poi scorrere linearmente rimuovendo consecutivi uguali.

2.5.27 Build-Max-Heap: indice decrescente

R: Le foglie sono già heap. MaxHeapify richiede figli sistemati: si parte dal basso.

2.5.28 Visita inorder con pila

R: Visita in ordine (inorder), $O(n)$. Pattern: push-sinistra, pop-visita-destra.

2.5.29 LCS senza tabella b

```
PRINT-LCS(c, X, Y, i, j)
if c[i,j] == 0 then return
if X[i] == Y[j]
PRINT-LCS(c, X, Y, i-1, j-1)
print X[i]
else if c[i-1,j] >= c[i,j-1]
PRINT-LCS(c, X, Y, i-1, j)
else
PRINT-LCS(c, X, Y, i, j-1)
```

Costo $O(m + n)$: ogni chiamata riduce i o j di 1.

2.5.30 Cammino minimo vs TSP

R: Cammino minimo: sorgente $\rightarrow$ destinazione, polinomiale (Dijkstra). TSP: circuito visitando ogni nodo una volta, NP-difficile.

2.5.31 Prima e ultima cifra (Java)

```java
int last  = n % 10;
int first = n / (int) Math.pow(10, (int)(Math.log10(n)));
```

2.5.32 `T extends Comparable`

R: La ricerca binaria usa confronti: serve `compareTo()`, disponibile solo se T estende `Comparable`.

2.5.33 Ereditarietà e attributo omonimo

R: `super.getIdA()` per `A.idA`, `this.getIdA()` per `B.idA`. Le variabili private non sono ereditate.

2.5.34 Errore in makeCombination

R: `values = numbers` copia solo il riferimento locale. Fix: copia elemento per elemento.

2.5.35 Eccezione in try-with-resources

R: L'eccezione di `close()` è soppressa (JLS 14.20.3): diventa *suppressed exception* della prima.

2.5.36 Downcasting Java

D: `A value = new B(); B value1 = (A) value; value = null;`?
R: `value` $\rightarrow$ null; `value1` punta all'oggetto `B` ancora in memoria.

2.5.37 Stack con due code

Push i-esimo: enqueue in ausiliaria, svuota corrente nell'ausiliaria, scambia. Costo: $O(i)$. Totale n push: $O(n^2)$.
Pop: dequeue. Costo: $O(1)$.

2.5.38 Ricerca in matrice destrorsa

R: Ricerca binaria su ogni riga. Complessità: $O(n \log n)$.

```java
public static <T extends Comparable<T>>
int[] search(T[][] M, T x) {
    int n = M.length;
    for (int i = 0; i < n; i++) {
        int lo = 0, hi = n - 1;
        while (lo <= hi) {
            int mid = (lo + hi) / 2;
            int cmp = M[i][mid].compareTo(x);
            if      (cmp == 0) return new int[]{i, mid};
            else if (cmp <  0) lo = mid + 1;
            else               hi = mid - 1;
        }
    }
    return null;
}
```